МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»
Навчально-науковий інститут телекомунікаційних систем
Кафедра _____

ЗВІТ

з лабораторних робіт № 1–2

з дисципліни «Розробка баз даних»

Тема: «Аналіз предметної області. Проєктування моделі даних. Створення моделі даних в середовищі СУБД»

Предметна область: «STEM-студія для дітей "Стемік"»

(система обліку занять, відвідувань, оплат і 3D-друку)

Виконав(ла):
студент(ка) групи _____
_____
(ПІБ)

Перевірив(ла):
_____

Оцінка: _____ Підпис: _____

Київ – 2026

ЗМІСТ

ЛАБОРАТОРНА РОБОТА № 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ. ПРОЄКТУВАННЯ МОДЕЛІ ДАНИХ.....3
1.1 Текстовий опис предметної області.....3
1.2 Сутності предметної області та їх атрибути.....4
1.3 Зв'язки між сутностями.....9
1.4 ER-діаграма предметної області.....10
1.5 Нормалізація моделі даних.....13
Висновки до лабораторної роботи № 1.....15
ЛАБОРАТОРНА РОБОТА № 2. СТВОРЕННЯ МОДЕЛІ ДАНИХ В СЕРЕДОВИЩІ СУБД.....17
2.1 Вибір СУБД та інструментів.....17
2.2 Перетворення ER-моделі на фізичну модель.....17
2.3 Обґрунтування вибору типів даних.....20
2.4 Первинні ключі.....21
2.5 Обмеження цілісності.....21
2.6 Правила каскадних дій.....21
2.7 DDL-скрипт.....22
2.8 Реалізація бази даних у СУБД.....24
Висновки до лабораторної роботи № 2.....30
ДОДАТОК А. Повний DDL-скрипт фізичної схеми.....32

ЛАБОРАТОРНА РОБОТА № 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ. ПРОЄКТУВАННЯ МОДЕЛІ ДАНИХ

Мета: формування практичних навичок аналізу предметної області та побудови концептуальної й логічної моделі даних, яка надалі буде реалізована засобами системи управління базами даних (СУБД).

Вихідні дані. Як предметну область обрано реальний проєкт автора – інформаційну систему STEM-студії для дітей «Стемік» (репозиторій `ut2ucn-dev/ut2ucn-dev.github.io`). Система складається з вебзастосунку для персоналу (`index.html`), мобільного/настільного застосунку на Flutter (`mobile/`), кабінету батьків (`client/`) та бази даних PostgreSQL у хмарному сервісі Supabase, схема якої еволюціонувала через 21 скрипт міграцій (`sql/0001…0021`). У роботі цю схему проаналізовано, а модель даних – спроєктовано наново за правилами нормалізації.

1.1 Текстовий опис предметної області

STEM-студія «Стемік» проводить для дітей 6–14 років заняття з 3D-моделювання (Tinkercad, 3D Makers), віртуальної реальності (VR-ігри, розробка у Delightex) та робототехніки зі штучним інтелектом (Ukit, UGot). Заняття відбуваються в чотирьох кабінетах (Клас AI, Клас VR, Комп'ютерний клас, Клас 3D) або онлайн, у форматі групи чи індивідуально. Кожне заняття належить до певного курсу, має дату, час, тривалість, тему та закріплюється за викладачем; якщо викладач хворіє, заняття веде інший співробітник на заміні. Регулярні заняття створюються серіями (щотижня в той самий час).

Адміністратор веде облік учнів (ПІБ, дата народження, контакти, примітки) і їхніх батьків. На кожному занятті викладач або адміністратор відмічає присутність учня: «П» (присутній), «Н» (відсутній) чи «ВПП» (відсутній з поважної причини) – та спосіб оплати: карткою, готівкою, сертифікатом або абонементом. Абонемент – це пакет із 4 занять одного курсу; при відвідуванні з абонемента списується одне заняття, а при «ВПП» воно повертається. Ціна заняття для клієнта та заробітна плата (ЗП)

викладача визначаються тарифом курсу залежно від формату й кількості дітей і «заморожуються» на занятті, щоб зміна тарифів не змінювала історію.

На заняттях з 3D діти створюють моделі, які студія друкує на 3D-принтерах. Для кожної роботи зберігається фото, ознака «пробного» заняття, факт друку, використаний принтер і пластик (філамент), вага моделі та час друку – це дає змогу розрахувати собівартість друку (матеріал, електроенергія, амортизація). Батьки мають власний кабінет, у який входять за номером телефону і паролем та бачать розклад, відвідування, абонементи і роботи своїх дітей. Викладачі можуть залишати коментарі про учнів.

Основні бізнес-процеси:

– планування розкладу занять (разові й серійні), призначення заміни викладача;

– реєстрація учнів і прив'язка їх до батьківських кабінетів;

– запис учнів на заняття, відмітка відвідування та оплати;

– продаж абонементів і списання з них занять;

– облік робіт учнів і їх 3D-друку, розрахунок собівартості;

– нарахування ЗП викладачам і формування звітів (оплачено / борг / невикористані абонементи).

Вимоги до системи: зберігати дані про співробітників, курси, тарифи, учнів, батьків, заняття, відвідування, абонементи, роботи, принтери й пластик; не допускати оплати за відсутнього учня, від'ємних цін, дублювання телефонів батьків і повторної відмітки учня на тому самому занятті; забороняти видалення даних, що мають фінансову історію; формувати звіти з оплат, боргів і ЗП.

1.2 Сутності предметної області та їх атрибути

Іменники з опису (студія, курс, заняття, учень, батьки, викладач, абонемент, робота, принтер, пластик, коментар, тариф…) є кандидатами на сутності. Не стали самостійними сутностями: «кабінет» і «формат заняття» – це атрибути заняття з фіксованим переліком значень (домен), «оплата» –

атрибути відвідування (у студії оплата завжди відноситься до конкретного відвідування), «студія» – єдиний екземпляр, її параметри виділено в службову таблицю налаштувань на фізичному рівні. Результат – 13 сутностей і 2 асоціативні сутності (табл. 1.1).

Таблиця 1.1 – Перелік сутностей

№	Сутність	Первинний ключ	Таблиця в проєкті	Призначення
1	Співробітник	employee_id	profiles	Викладачі й адміністратори, ролі доступу
2	Курс	course_id	— (текст у lessons.course)	Напрямок навчання, вікова група
3	Тариф	tariff_id	rate_cards	ЗП викладача та ціна заняття для курсу і формату
4	Учень	student_id	students	Діти, що навчаються у студії
5	Батьки	parent_id	parent_accounts	Власник батьківського кабінету
6	Облікові дані	parent_id	parent_credentials	Пароль і блокування входу (1:1 з Батьками)
7	Заняття	lesson_id	lessons	Заняття в розкладі
8	Запис на заняття*	(lesson_id, student_id)	масив lessons.student_ids	Які учні записані на заняття
9	Відвідування	attendance_id	lesson_attendance	П/Н/ВПП та оплата учня на занятті
10	Абонемент	subscription_id	subscriptions	Пакет з N занять курсу
11	Робота учня	work_id	works	Модель учня, її фото та друк
12	3D-принтер	printer_id	printers	Обладнання для друку
13	Філамент	filament_id	filaments	Пластик для друку
14	Коментар	comment_id	student_comments	Нотатки викладача про учня
15	Батьки–Учень*	(parent_id, student_id)	parent_students	Які діти видимі в кабінеті батьків

* – асоціативні сутності, що реалізують зв'язки M:N.

Склад атрибутів кожної сутності наведено в таблицях 1.2–1.14. На цьому етапі вказано логічний тип (домен) атрибута; конкретні типи СУБД обрано в лабораторній роботі № 2.

Таблиця 1.2 – Атрибути сутності «Співробітник»

Атрибут	Тип (домен)	Опис
employee_id (PK)	ідентифікатор	Унікальний ідентифікатор співробітника
email	текст	Логін, унікальний
last_name	текст	Прізвище
first_name	текст	Ім'я
role	перелік	teacher / admin / superadmin
active	логічний	Працює / звільнений (м'яке видалення)
photo_url	текст	Посилання на фото

Таблиця 1.3 – Атрибути сутності «Курс»

Атрибут	Тип (домен)	Опис
course_id (PK)	ідентифікатор	Унікальний ідентифікатор курсу
name	текст	Назва, напр. «Robo AI (UGot 9-12)»
direction	перелік	3D / VR / ROBO
age_min	ціле	Мінімальний вік
age_max	ціле	Максимальний вік
online_print_price	гроші	Доплата за друк на онлайн-занятті

Таблиця 1.4 – Атрибути сутності «Тариф»

Атрибут	Тип (домен)	Опис
tariff_id (PK)	ідентифікатор	Унікальний ідентифікатор
course_id (FK)	ідентифікатор	Курс
tariff_type	перелік	group_1, group_2_6, individual, individual_online, group_online, extra_service
teacher_pay	гроші	ЗП викладача за заняття
lesson_price	гроші	Ціна заняття для клієнта
subscription_price	гроші	Ціна абонемента в цьому форматі

Таблиця 1.5 – Атрибути сутності «Учень»

Атрибут	Тип (домен)	Опис
student_id (PK)	ідентифікатор	Унікальний ідентифікатор учня
last_name	текст	Прізвище
first_name	текст	Ім'я
birthday	дата	Дата народження
phone	текст	Контактний телефон
notes	текст	Примітки (алергії, особливості)
active	логічний	Навчається / вибув

Таблиця 1.6 – Атрибути сутності «Батьки»

Атрибут	Тип (домен)	Опис
parent_id (PK)	ідентифікатор	Унікальний ідентифікатор
phone	текст	Логін кабінету (лише цифри, 380…), унікальний
last_name	текст	Прізвище
first_name	текст	Ім'я
active	логічний	Кабінет увімкнено
last_login_at	дата-час	Останній вхід

Таблиця 1.7 – Атрибути сутності «Облікові дані»

Атрибут	Тип (домен)	Опис
parent_id (PK, FK)	ідентифікатор	Батьки (зв'язок 1:1)
password_hash	текст	Хеш пароля (bcrypt)
password_set_at	дата-час	Коли задано пароль
failed_attempts	ціле	Невдалі спроби входу поспіль
locked_until	дата-час	Блокування після перебору паролів

Таблиця 1.8 – Атрибути сутності «Заняття»

Атрибут	Тип (домен)	Опис
lesson_id (PK)	ідентифікатор	Унікальний ідентифікатор
course_id (FK)	ідентифікатор	Курс
teacher_id (FK)	ідентифікатор	Викладач
substitute_id (FK)	ідентифікатор	Викладач на заміні (необов'язковий)
series_id	ідентифікатор	Серія щотижневих занять
lesson_date, start_time	дата, час	Коли відбувається
duration_minutes	ціле	Тривалість
format	перелік	group / individual / individual_online / group_online
room	перелік	Кабінет
theme	текст	Тема заняття
completed	логічний	Проведено
payroll_amount, price_amount	гроші	«Заморожені» ЗП і ціна

Таблиця 1.9 – Атрибути сутності «Відвідування»

Атрибут	Тип (домен)	Опис
attendance_id (PK)	ідентифікатор	Унікальний ідентифікатор
lesson_id, student_id (FK)	ідентифікатор	Запис на заняття
status	перелік	present / absent / excused (П / Н / ВПП)
payment_method	перелік	card / cash / subscription / certificate
amount	гроші	Сплачена сума
discount_percent	відсоток	Ручна знижка

Атрибут	Тип (домен)	Опис
subscription_id (FK)	ідентифікатор	Абонемент, з якого списано заняття
print_enabled, print_price	логічний, гроші	Доплата за друк

Таблиця 1.10 – Атрибути сутності «Абонемент»

Атрибут	Тип (домен)	Опис
subscription_id (PK)	ідентифікатор	Унікальний ідентифікатор
student_id (FK)	ідентифікатор	Учень
course_id (FK)	ідентифікатор	Курс
total_lessons	ціле	Занять у пакеті (4)
remaining_lessons	ціле	Залишок занять
total_price	гроші	Вартість
payment_method	перелік	card / cash
status	перелік	active / completed / cancelled
purchased_at	дата	Дата купівлі

Таблиця 1.11 – Атрибути сутності «Робота учня»

Атрибут	Тип (домен)	Опис
work_id (PK)	ідентифікатор	Унікальний ідентифікатор
lesson_id, student_id (FK)	ідентифікатор	Запис на заняття
photo_url	текст	Фото моделі
is_trial	логічний	Пробне заняття
printed	логічний	Надруковано
printer_id, filament_id (FK)	ідентифікатор	Принтер і пластик
weight_g	дійсне	Вага моделі, г
print_minutes	ціле	Час друку, хв

Таблиця 1.12 – Атрибути сутності «3D-принтер»

Атрибут	Тип (домен)	Опис
printer_id (PK)	ідентифікатор	Унікальний ідентифікатор
name	текст	Модель, напр. Bambu Lab P1S
power_watts	дійсне	Потужність, Вт
depreciation_per_hour	гроші	Амортизація за годину
active	логічний	У роботі

Таблиця 1.13 – Атрибути сутності «Філамент»

Атрибут	Тип (домен)	Опис
filament_id (PK)	ідентифікатор	Унікальний ідентифікатор
material	текст	PLA, PETG…
color	текст	Колір
price_per_kg	гроші	Ціна за кг
active	логічний	Є в наявності

Таблиця 1.14 – Атрибути сутності «Коментар»

Атрибут	Тип (домен)	Опис
comment_id (PK)	ідентифікатор	Унікальний ідентифікатор
student_id (FK)	ідентифікатор	Учень
author_id (FK)	ідентифікатор	Автор-співробітник
body	текст	Текст коментаря
created_at	дата-час	Час створення

1.3 Зв'язки між сутностями

Для кожної пари пов'язаних сутностей визначено кардинальність і клас належності (обов'язковість) – табл. 1.15. Зв'язки M:N «Заняття – Учень» та «Батьки – Учень» на логічному рівні замінено асоціативними сутностями «Запис на заняття» та «Батьки–Учень».

Таблиця 1.15 – Зв'язки між сутностями

Сутність А	Сутність Б	Кардинальність	Обов'язковість
Співробітник	Заняття (веде)	1:N	обов'язковий з боку Заняття (у кожного заняття є викладач)
Співробітник	Заняття (заміняє)	1:N	опціональний з обох боків
Курс	Тариф	1:N	обов'язковий з обох боків (курс без тарифів не має ціни)
Курс	Заняття	1:N	обов'язковий з боку Заняття
Заняття	Учень	M:N (через Запис на заняття)	опціональний (заняття може бути без учнів)
Запис на заняття	Відвідування	1:1	опціональний з боку Відвідування (ще не відмічено)
Запис на заняття	Робота учня	1:1	опціональний (робота є не на кожному занятті)
Учень	Абонемент	1:N	обов'язковий з боку Абонемента
Курс	Абонемент	1:N	обов'язковий з боку Абонемента
Абонемент	Відвідування	1:N	опціональний (лише при оплаті абонементом)
Батьки	Учень	M:N (через Батьки–Учень)	обов'язковий з боку Батьків, опціональний з боку Учня
Батьки	Облікові дані	1:1	опціональний (пароль ще не видано)
3D-принтер	Робота учня	1:N	опціональний
Філамент	Робота учня	1:N	опціональний

Сутність А	Сутність Б	Кардинальність	Обов'язковість
Учень	Коментар	1:N	обов'язковий з боку Коментаря
Співробітник	Коментар	1:N	опціональний (автор може бути видалений)

Приклад зв'язку, обов'язкового лише з одного боку: кожне Заняття обов'язково має Викладача, тоді як Співробітник (наприклад, адміністратор) може не вести жодного заняття. Службові зв'язки «хто вніс запис» (marked_by, purchased_by, created_by, printed_by, updated_by) мають однаковий характер (Співробітник 1:N, опціонально) і для наочності показані лише на фізичній діаграмі (ЛР № 2).

1.4 ER-діаграма предметної області

Концептуальну модель побудовано в нотації Чена (рис. 1.1): сутності – прямокутники, зв'язки – ромби з підписаною кардинальністю, асоціативна сутність – прямокутник із подвійними бічними межами. Товста лінія позначає обов'язкову участь (total participation), тонка – опціональну. Атрибути сутностей винесено в таблиці 1.2–1.14, щоб не перевантажувати схему.

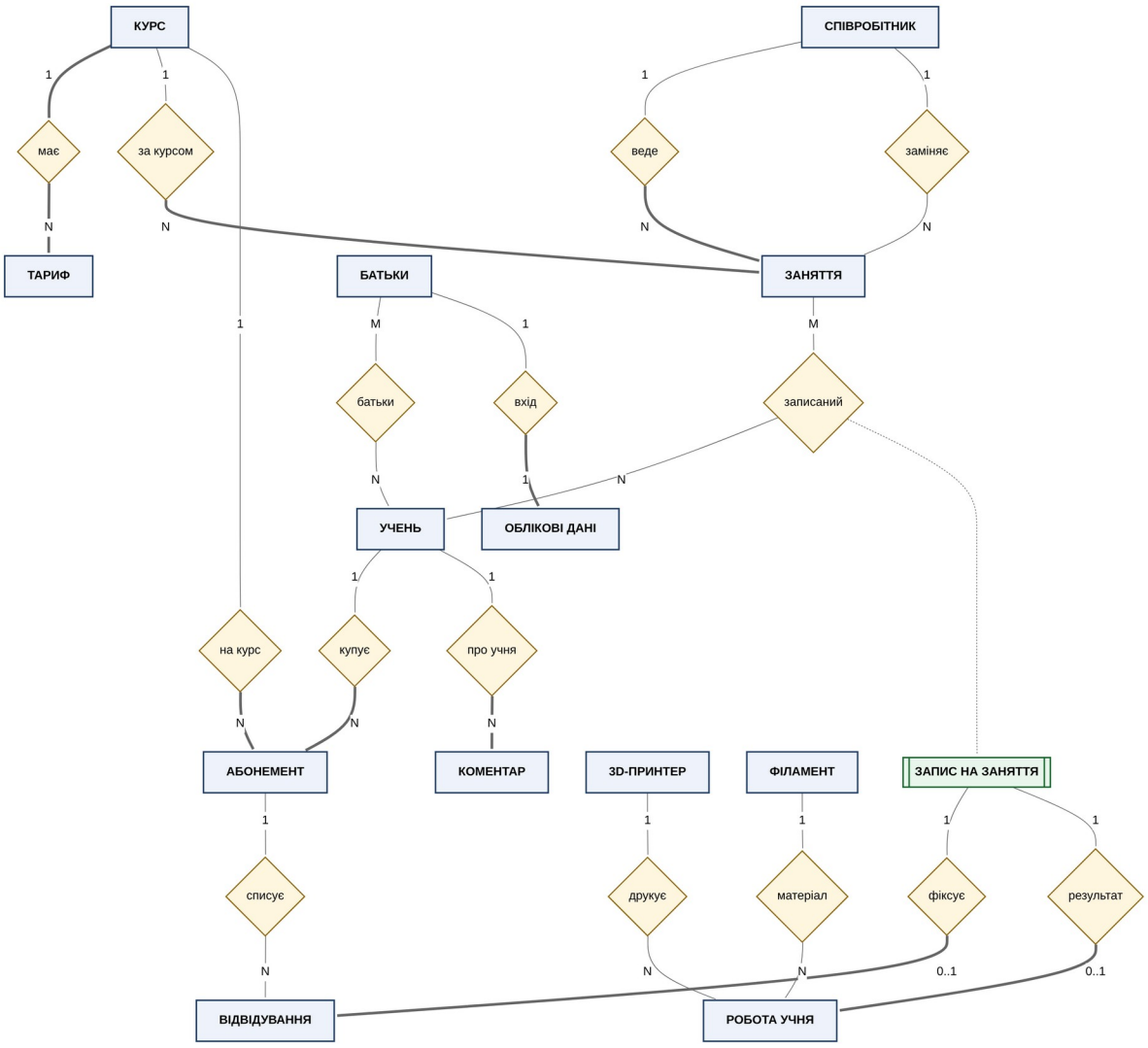


Рисунок 1.1 – Концептуальна ER-діаграма предметної області (нотація Чена)

Логічну модель з ключовими атрибутами та зв'язками в нотації «Crow's Foot» наведено на рис. 1.2. Символи на кінцях ліній: риска – «один», кружечок – «опціонально», «пташина лапка» – «багато». Діаграми побудовано CASE-засобом Mermaid (вихідні тексти – у каталозі `docs/db-labs/diagrams`).

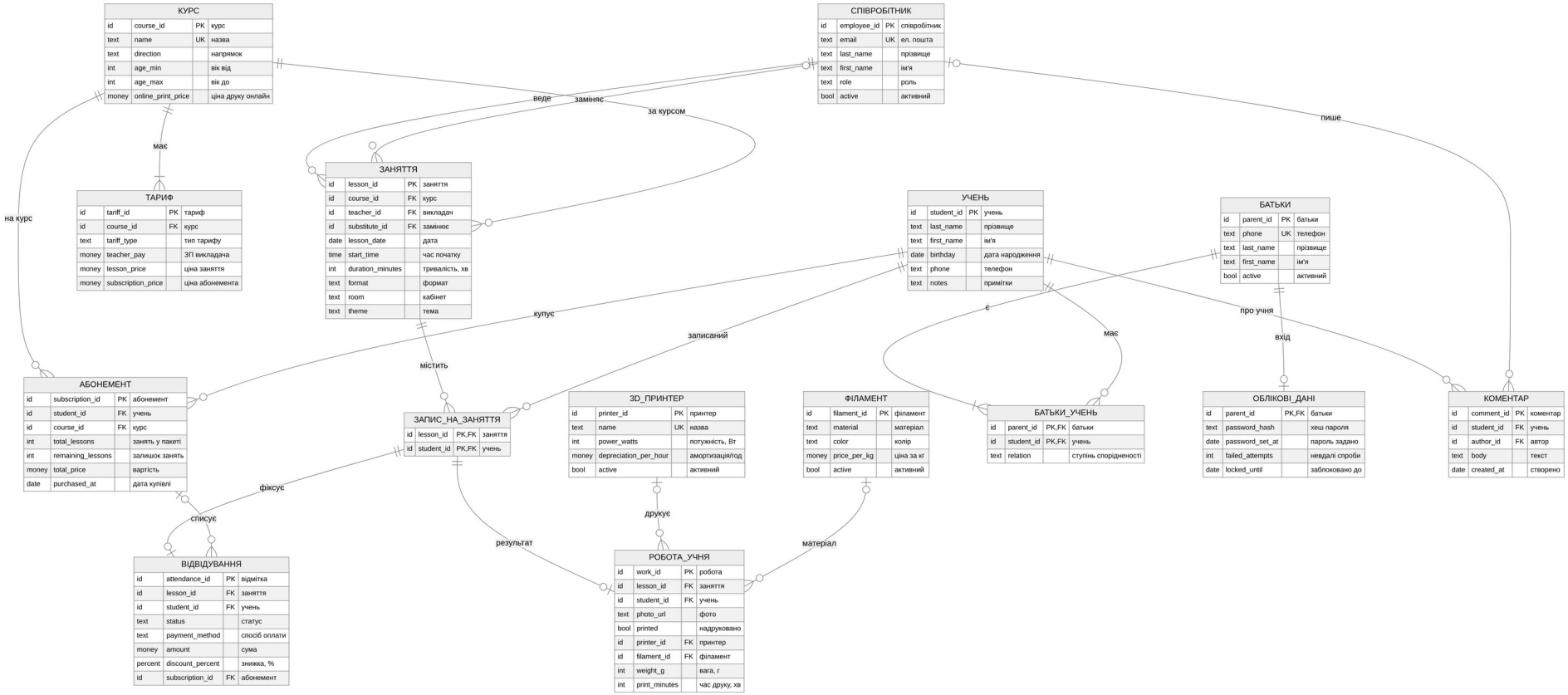


Рисунок 1.2 – Логічна ER-діаграма (нотація Crow's Foot) з PK/FK

1.5 Нормалізація моделі даних

Аналіз робочої схеми проєкту виявив кілька відхилень від нормальних форм, що виникли під час швидкого розвитку застосунку. У спроєктованій моделі їх усунено (табл. 1.16).

Перша нормальна форма (1НФ)

Порушення 1. У таблиці `lessons` список учнів зберігається масивом `student_ids uuid[]` – кілька значень в одному полі (повторювана група). Через це неможливо оголосити зовнішній ключ на учня, а при видаленні учня проєкту знадобився окремий тригер `cleanup_deleted_student_from_lessons` (міграція 0007), щоб вичищати його ідентифікатор із масивів. **Усунення:** асоціативна сутність Запис на заняття(`lesson_id`, `student_id`) зі складеним первинним ключем.

Порушення 2. `students.name` містить прізвище та ім'я в одному полі (неатомарне значення); пошук і сортування за прізвищем вимагають розбору рядка. **Усунення:** окремі атрибути `last_name`, `first_name`.

Порушення 3. Таблиця `rate_cards` має понад 30 стовпців виду `zp_group_1, price_group_1, zp_individual, price_individual, …` – це повторювана група «формат → (ЗП, ціна)», розгорнута в стовпці. Додавання нового формату вимагало зміни структури таблиці (міграції 0002, 0016), а старі стовпці залишалися «мертвими». **Усунення:** сутність Тариф(`course_id`, `tariff_type`, `teacher_pay`, `lesson_price`, `subscription_price`) – один рядок на пару «курс × формат».

Друга нормальна форма (2НФ)

2НФ стосується таблиць зі складеним ключем. У моделі їх дві: Запис на заняття(`lesson_id`, `student_id`) та Батьки–Учень(`parent_id`, `student_id`). Якби в запис на заняття додали, наприклад, `student_name` чи `lesson_date`, ці атрибути залежали б лише від частини ключа (`student_id` або `lesson_id`) – часткова залежність. Тому в асоціативних таблицях залишено лише

атрибути самого зв'язку (relation – ступінь спорідненості, яка залежить від пари «батько–дитина» цілком), а все інше отримується з’єднанням із сутностями Учень і Заняття.

Окремо проаналізовано Відвідування: природним ключем є пара (lesson_id, student_id), а атрибути status, payment_method, amount залежать саме від пари (той самий учень на іншому занятті має іншу відмітку). Сума ж «ціна заняття» залежить лише від lesson_id, тому зберігається в Занятті (price_amount), а не у Відвідуванні – часткову залежність не допущено.

Третя нормальна форма (3НФ)

Порушення 1. У lesson_attendance є стовпець owner_id (викладач), хоча викладач однозначно визначається заняттям: attendance_id → lesson_id → owner_id – транзитивна залежність. При зміні викладача заняття проєкт змушений оновлювати owner_id одразу у трьох таблицях (lessons, works, lesson_attendance – див. mobile/lib/data/supabase_api.dart), інакше виникає аномалія оновлення. **Усунення:** owner_id прибрано з Відвідування і Роботи; викладача отримують через Заняття. (У робочому проєкті денормалізацію залишено свідомо – для простих політик доступу Row Level Security, де викладач бачить «свої» рядки.)

Порушення 2. У works зберігається printed_by_name – ім'я того, хто надрукував модель: work_id → printed_by → ПІБ. Після зміни прізвища співробітника старі записи показуватимуть неактуальне ім'я. **Усунення:** зовнішній ключ printed_by на Співробітника.

Порушення 3. students.parent_last_name, parent_first_name дублюють дані Батьків (parent_accounts.name), а телефон батьків використовується і як телефон учня. **Усунення:** дані батьків зберігаються лише в сутності Батьки і пов'язуються з учнем через Батьки–Учень.

Порушення 4. Назва курсу (lessons.course, subscriptions.course) дублюється текстом у кожному рядку, а характеристики курсу (вік,

напрямок) ніде не зберігаються. **Усунення:** окрема сутність Курс і зовнішні ключі `course_id`.

Свідомо збережені похідні атрибути. `payroll_amount` і `price_amount` у Занятті та `amount` у Відвідуванні формально можна обчислити з тарифу, але вони фіксують ціну *на момент проведення заняття*: тариф змінюється з часом, а історія оплат і ЗП змінюватися не повинна. Отже, вони залежать від ключа заняття (це історичний факт), а не від поточного рядка тарифу, і ЗНФ не порушують. Залишок абонемента `remaining_lessons` зберігається з тієї ж причини та контролюється обмеженням `CHECK`.

Таблиця 1.16 – Підсумок нормалізації

Таблиця проєкту	Проблема	НФ	Рішення в моделі
lessons.student_ids	масив значень в одному полі	1НФ	таблиця lesson_student
students.name	ПІБ в одному полі	1НФ	last_name + first_name
rate_cards (30+ стовпців)	повторювана група форматів	1НФ	таблиця tariff
lesson_attendance.owner_id	транзитивна залежність через lesson_id	3НФ	вилучено, через JOIN з lesson
works.owner_id, printed_by_name	транзитивна залежність	3НФ	вилучено / FK printed_by
students.parent_*_name	дублювання даних батьків	3НФ	сутність parent + parent_student
course як текст	дублювання, немає атрибутів курсу	3НФ	сутність course, FK course_id

Перевірка за правилом Кодда: у кожній таблиці моделі кожен неключовий атрибут залежить від ключа (1НФ – атомарність), від усього ключа (2НФ – для lesson_student, parent_student) і тільки від ключа (3НФ – після вилучення owner_id, printed_by_name, імен батьків і тексту курсу). Отже, модель перебуває в третій нормальній формі.

Висновки до лабораторної роботи № 1

У роботі проаналізовано предметну область STEM-студії «Стемік» на основі реального програмного проєкту: описано бізнес-процеси, об'єкти обліку та вимоги до системи; виділено 13 сутностей і 2 асоціативні сутності (кожна має не менше 5 атрибутів, окрім асоціативних), визначено 16 зв'язків

з кардинальністю та обов'язковістю. Побудовано концептуальну ER-діаграму в нотації Чена та логічну – в нотації Crow's Foot. Аналіз робочої схеми виявив порушення 1НФ (масив `student_ids`, широка таблиця тарифів) і 3НФ (дубльований `owner_id`, `printed_by_name`, імена батьків у картці учня); у новій моделі їх усунено, і модель приведено до 3НФ.

ЛАБОРАТОРНА РОБОТА № 2. СТВОРЕННЯ МОДЕЛІ ДАНИХ В СЕРЕДОВИЩІ СУБД

Мета: перехід від концептуальної ER-моделі, побудованої в лабораторній роботі № 1, до повноцінної фізичної (реляційної) моделі даних та її реалізація в конкретній СУБД.

2.1 Вибір СУБД та інструментів

Обрано СУБД **PostgreSQL 16** – ту саму, на якій працює робочий проєкт (хмарний сервіс Supabase). Це дає змогу надалі перенести спроєктовану схему в продуктивну базу. Скрипти виконувались у консольному клієнті `psql`; таблиці створено в окремій схемі `stemik`, щоб не змішувати їх із системними. Діаграму таблиць згенеровано автоматично з системного каталогу PostgreSQL (`information_schema`, `pg_constraint`) і візуалізовано засобом Mermaid. Усі файли роботи розміщено в репозиторії в каталозі `docs/db-labs/`:

– `sql/01_schema.sql` – DDL-скрипт фізичної схеми;

– `sql/02_seed.sql` – тестові дані;

– `sql/03_constraint_tests.sql` – перевірка обмежень цілісності й каскадних дій;

– `diagrams/*.mmd` – вихідні тексти діаграм, `img/` – рисунки звіту.

2.2 Перетворення ER-моделі на фізичну модель

Кожна сутність стала таблицею (назви – англійською в однині, `snake_case`, як прийнято в проєкті), кожен атрибут – стовпцем із конкретним типом. Зв'язки 1:N реалізовано зовнішнім ключем на боці «багато», зв'язки M:N – асоціативними таблицями `lesson_student` і `parent_student`, зв'язки 1:1 – зовнішнім ключем з обмеженням унікальності: у `parent_credential` FK одночасно є первинним ключем, а `attendance` і `work` посилаються на запис `lesson_student` складеним FK (`lesson_id`, `student_id`) з `UNIQUE`. Завдяки складеному FK неможливо відмітити відвідування чи зберегти роботу учня, який не записаний на це заняття. Додано службову таблицю

`studio_settings` (рівно один рядок завдяки `PRIMARY KEY` типу `BOOLEAN` з `CHECK (id)`). Отримана фізична схема з 16 таблиць наведена на рис. 2.1.

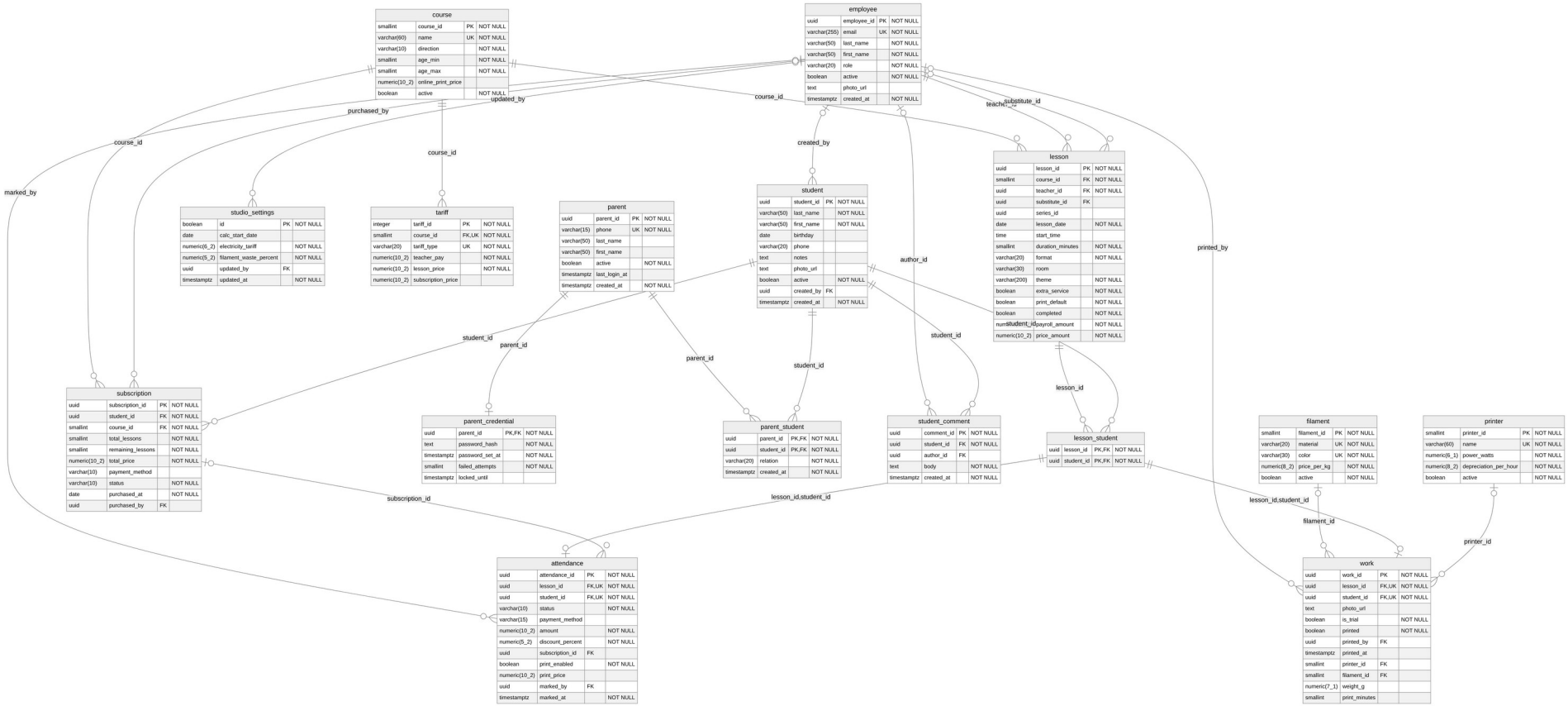


Рисунок 2.1 – Фізична схема бази даних (таблиці, типи, PK/FK/UK, зв'язки; у підписах зв'язків – стовпці FK)

Примітка: засіб візуалізації не допускає коми в назві типу, тому `numeric(10,2)` на діаграмі показано як `numeric(10_2)`.

2.3 Обґрунтування вибору типів даних

Типи обрано за правилом «найменший тип, що гарантовано вміщує весь очікуваний діапазон значень». Основні рішення наведено в табл. 2.1.

Таблиця 2.1 – Обґрунтування вибору типів даних

Таблиця	Атрибут	Тип	Обґрунтування
усі операційні	*_id (employee, student, lesson…)	UUID	Ідентифікатор генерується на клієнті в офлайн-режимі (застосунок має чергу змін outbox) і при синхронізації не конфліктує з іншими пристроями
course, tariff, printer, filament	*_id	SMALLINT / INT IDENTITY	Невеликі довідники (десятки рядків), створюються лише адміністратором онлайн
tariff, lesson, attendance, subscription	ціни, суми, ЗП	NUMERIC(10,2)	Гроші – лише точний тип; FLOAT дав би похибки округлення в звітах
attendance	discount_percent	NUMERIC(5,2)	0.00…100.00 %
employee, student	last_name, first_name	VARCHAR(50)	Достатньо для будь-якого прізвища
employee	email	VARCHAR(255)	Максимальна практична довжина адреси
parent	phone	VARCHAR(15)	Лише цифри (до 15 за E.164); число не підходить – втратиться провідний нуль
student	phone	VARCHAR(20)	Може містити +, пробіли, дужки
lesson	format, room	VARCHAR(20/30) + CHECK	Обмежений перелік значень (домен)
lesson	lesson_date / start_time	DATE / TIME	Дата і час окремо: є заняття без часу; сортування й порівняння засобами СУБД
lesson	duration_minutes	SMALLINT	15…240 хв – INT надлишковий
lesson	theme	VARCHAR(200)	Коротка тема заняття
student, student_comment	notes, body	TEXT	Довільний текст без чіткої межі
work	weight_g	NUMERIC(7,1)	Вага до 999999,9 г з точністю 0,1 г
усі	created_at, marked_at…	TIMESTAMPTZ	Момент часу з часовим поясом (сервер у UTC, користувачі – у Києві)
усі	active, printed, completed…	BOOLEAN	Рівно два значення

2.4 Первинні ключі

Для всіх основних таблиць використано **штучні (сурогатні) ключі**: вони стабільні, компактні й не змінюються протягом життя запису. Природні ключі збережено як обмеження `UNIQUE`: `employee.email`, `parent.phone`, `course.name`, `printer.name`, (`filament.material`, `color`), (`tariff.course_id`, `tariff_type`). Асоціативні таблиці мають **складені** первинні ключі з двох FK – (`lesson_id`, `student_id`) і (`parent_id`, `student_id`), що одночасно забороняє дублювання зв'язку. Для довідників застосовано автоінкремент `GENERATED ALWAYS AS IDENTITY` (стандарт SQL, аналог `SERIAL`), для операційних даних – `UUID DEFAULT gen_random_uuid()`.

2.5 Обмеження цілісності

Таблиця 2.2 – Обмеження цілісності (вибірка)

Вид	Де застосовано	Бізнес-правило
NOT NULL	прізвище, ім'я, email, дата заняття, курс і викладач заняття, статус відвідування	Без цих значень запис втрачає сенс
UNIQUE	employee.email, parent.phone, course.name, (lesson_id, student_id) в attendance і work	Один логін – один користувач; одна відмітка й одна робота учня на занятті
CHECK (перелік)	role, direction, tariff_type, format, room, status, payment_method	Лише допустимі значення домену
CHECK (діапазон)	ціни ≥ 0, discount 0…100, тривалість 15…240, вік 4…18, age_min ≤ age_max	Від'ємна ціна чи 0-хвилинне заняття неможливі
CHECK (між стовпцями)	remaining_lessons між 0 і total_lessons; substitute_id ≠ teacher_id	Залишок абонемента коректний; викладач не замінює сам себе
CHECK (бізнес-правило)	attendance_payment_only_present_chk	Оплата можлива лише для присутнього учня (Н/ВПП – без оплати)
CHECK (бізнес-правило)	attendance_subscription_payment_chk	Посилання на абонемент – лише при оплаті абонементом
CHECK (формат)	parent.phone ~ '^[0-9]{10,15}$'	Телефон-логін – лише цифри
DEFAULT	role = 'teacher', active = TRUE, total_lessons = 4, duration = 60, created_at = now(), purchased_at = CURRENT_DATE	Типові значення при створенні

2.6 Правила каскадних дій

Для кожного зовнішнього ключа визначено дію при видаленні батьківського запису (табл. 2.3). При оновленні для всіх FK обрано `ON UPDATE CASCADE`: сурогатні ключі не змінюються, тож це лише страховка.

Таблиця 2.3 – Обґрунтування каскадних дій ON DELETE

Зовнішній ключ	ON DELETE	Обґрунтування
lesson.course_id → course	RESTRICT	Не можна видалити курс, за яким проводились заняття – зламається історія
lesson.teacher_id → employee	RESTRICT	Заняття – основа нарахування ЗП; співробітника «звільняють» через active = false
lesson.substitute_id → employee	SET NULL	Заміна опціональна: заняття залишається за основним викладачем
lesson_student.lesson_id → lesson	CASCADE	Запис учня без заняття не має сенсу
lesson_student.student_id → student	RESTRICT	Учень з історією занять не видаляється, лише деактивується (active = false)
attendance(lesson_id, student_id) → lesson_student	CASCADE	Відмітка залежить від запису; видалення помилково створеного заняття видаляє і відмітки
attendance.subscription_id → subscription	RESTRICT	Абонемент, з якого списано заняття, – фінансовий документ
subscription.student_id → student	RESTRICT	Абонементи – фінансова історія
subscription.course_id → course	RESTRICT	Те саме
tariff.course_id → course	CASCADE	Тарифи не існують без курсу
work(lesson_id, student_id) → lesson_student	CASCADE	Робота прив'язана до запису на заняття
work.printer_id / filament_id	SET NULL	Списання принтера чи пластику не повинно видаляти роботи учнів
parent_student.* → parent, student	CASCADE	Рядок зв'язку без однієї зі сторін не має сенсу
parent_credential.parent_id → parent	CASCADE	Пароль існує лише разом із кабінетом
student_comment.student_id → student	CASCADE	Коментарі – частина картки учня
*.created_by / marked_by / purchased_by / printed_by / author_id / updated_by → employee	SET NULL	Службова інформація «хто вніс запис»; сам запис має зберегтися

Відмінність від робочого проєкту: у Supabase-схемі для `subscriptions.student_id` і `lesson_attendance.student_id` встановлено `CASCADE`. У навчальній моделі обрано `RESTRICT` + м'яке видалення (`student.active`), як рекомендовано для фінансово-облікових документів.

2.7 DDL-скрипт

Таблиці створюються в порядку залежностей: спочатку довідники та «батьківські» таблиці (employee, course, parent, student), потім ті, що на них посилаються, асоціативні таблиці – після обох сторін зв'язку. Фрагмент

скрипту для центральних таблиць наведено нижче; повний скрипт – у додатку А.

```
CREATE TABLE lesson (
    lesson_id        UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    course_id        SMALLINT      NOT NULL REFERENCES course(course_id)
                     ON DELETE RESTRICT ON UPDATE CASCADE,
    teacher_id       UUID          NOT NULL REFERENCES employee(employee_id)
                     ON DELETE RESTRICT ON UPDATE CASCADE,
    substitute_id    UUID          REFERENCES employee(employee_id)
                     ON DELETE SET NULL ON UPDATE CASCADE,
    series_id        UUID,
    lesson_date      DATE          NOT NULL,
    start_time       TIME,
    duration_minutes SMALLINT      NOT NULL DEFAULT 60
                     CHECK (duration_minutes BETWEEN 15 AND 240),
    format           VARCHAR(20)   NOT NULL DEFAULT 'group'
                     CHECK (format IN ('group', 'individual',
                                       'individual_online', 'group_online')),
    room             VARCHAR(30)   CHECK (room IN ('Клас AI', 'Клас VR',
                                                   'Комп''ютерний клас', 'Клас 3D')),
    theme            VARCHAR(200)  NOT NULL DEFAULT '',
    extra_service    BOOLEAN       NOT NULL DEFAULT FALSE,
    print_default    BOOLEAN       NOT NULL DEFAULT FALSE,
    completed        BOOLEAN       NOT NULL DEFAULT FALSE,
    payroll_amount   NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (payroll_amount >= 0),
    price_amount     NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (price_amount >= 0),
    CONSTRAINT lesson_substitute_chk CHECK (substitute_id IS NULL OR substitute_id <>
teacher_id)
);

CREATE TABLE lesson_student (
    lesson_id   UUID NOT NULL REFERENCES lesson(lesson_id)
                ON DELETE CASCADE ON UPDATE CASCADE,
    student_id  UUID NOT NULL REFERENCES student(student_id)
                ON DELETE RESTRICT ON UPDATE CASCADE,
    PRIMARY KEY (lesson_id, student_id)
);

CREATE TABLE attendance (
    attendance_id    UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    lesson_id        UUID          NOT NULL,
    student_id       UUID          NOT NULL,
    status           VARCHAR(10)   NOT NULL
                     CHECK (status IN ('present', 'absent', 'excused')),
    payment_method   VARCHAR(15)   CHECK (payment_method IN
                                   ('card', 'cash', 'subscription', 'certificate')),
    amount           NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (amount >= 0),
    discount_percent NUMERIC(5,2)  NOT NULL DEFAULT 0
                     CHECK (discount_percent BETWEEN 0 AND 100),
    subscription_id  UUID          REFERENCES subscription(subscription_id)
                     ON DELETE RESTRICT ON UPDATE CASCADE,
    print_enabled    BOOLEAN       NOT NULL DEFAULT FALSE,
    print_price      NUMERIC(10,2) CHECK (print_price >= 0),
    marked_by        UUID          REFERENCES employee(employee_id)
                     ON DELETE SET NULL ON UPDATE CASCADE,
    marked_at        TIMESTAMPTZ   NOT NULL DEFAULT now(),
    UNIQUE (lesson_id, student_id),
    FOREIGN KEY (lesson_id, student_id)
        REFERENCES lesson_student(lesson_id, student_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    -- оплата можлива лише для присутнього учня
    CONSTRAINT attendance_payment_only_present_chk CHECK (status = 'present' OR
(payment_method IS NULL AND amount = 0)),
    -- посилання на абонемент — лише при оплаті абонементом
    CONSTRAINT attendance_subscription_payment_chk CHECK (subscription_id IS NULL OR
payment_method = 'subscription')
);
```

2.8 Реалізація бази даних у СУБД

Скрипти виконано в PostgreSQL 16.15. Результати показано на скриншотах консолі `psql` (рис. 2.2–2.11).

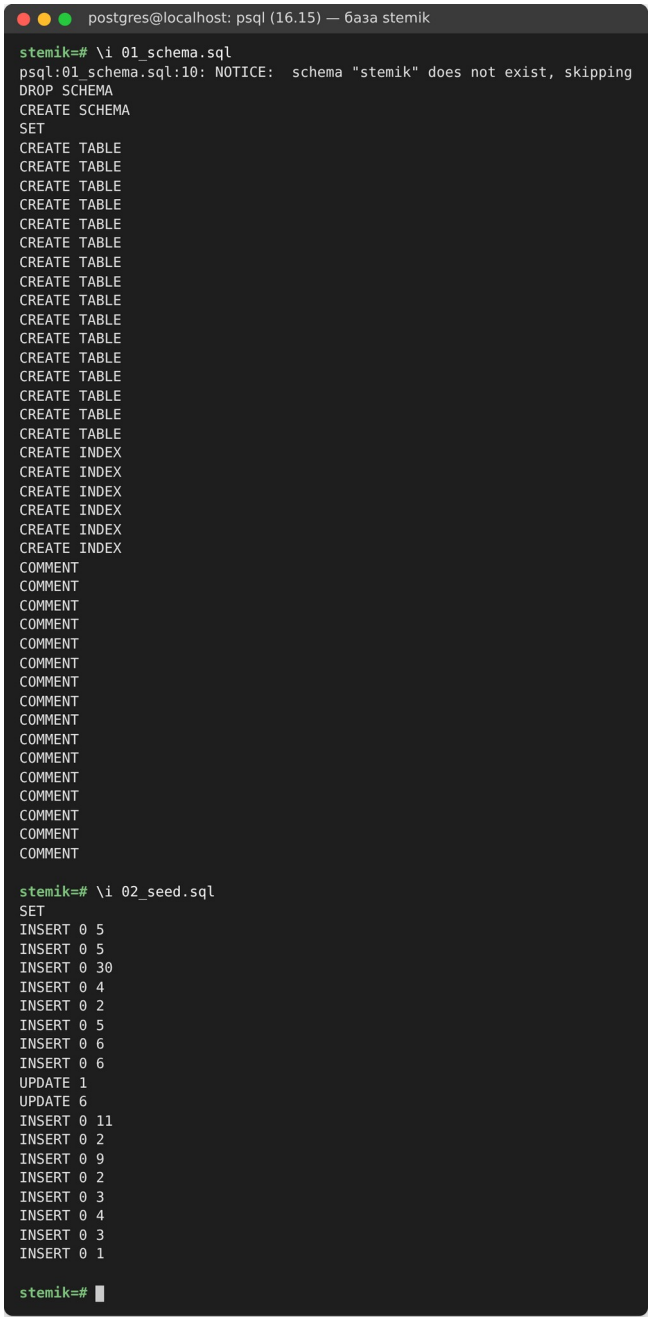


Рисунок 2.2 – Виконання DDL-скрипту та завантаження тестових даних

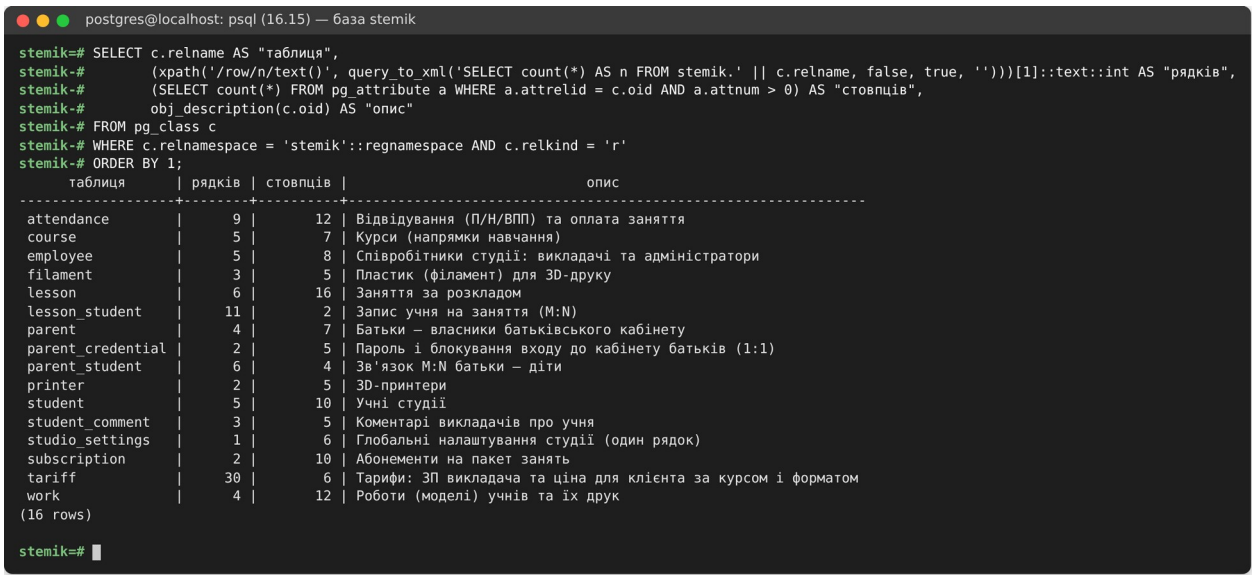


Рисунок 2.3 – Перелік таблиць схеми stemik: кількість рядків, стовпців і опис

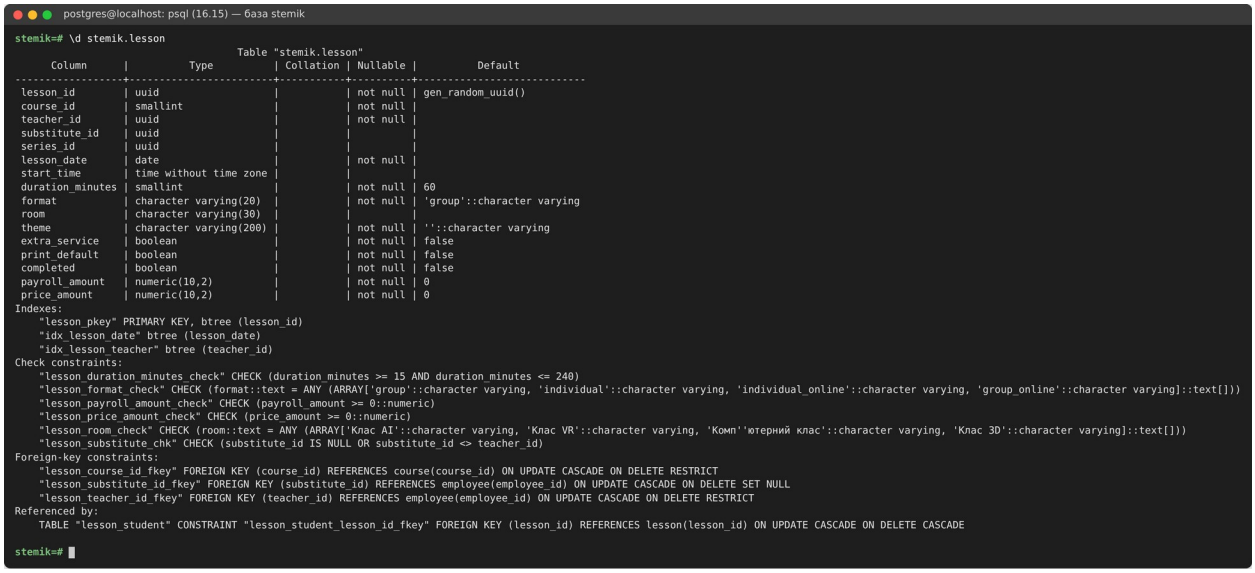


Рисунок 2.4 – Структура таблиці lesson (\d lesson): стовпці, типи, NOT NULL, DEFAULT, індекси, обмеження і FK

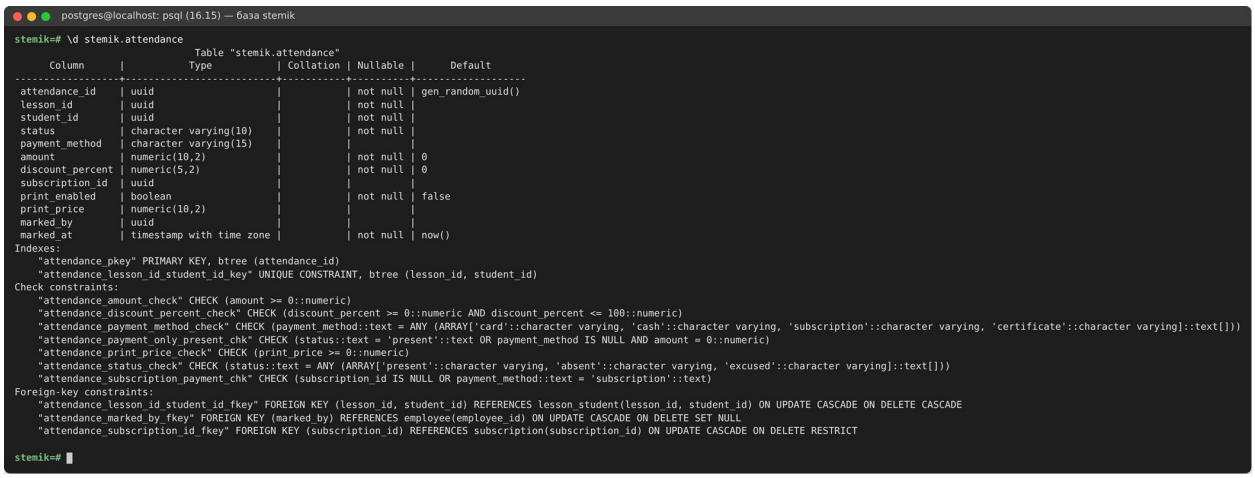


Рисунок 2.5 – Структура таблиці attendance: складений FK на lesson_student та CHECK-обмеження бізнес-правил

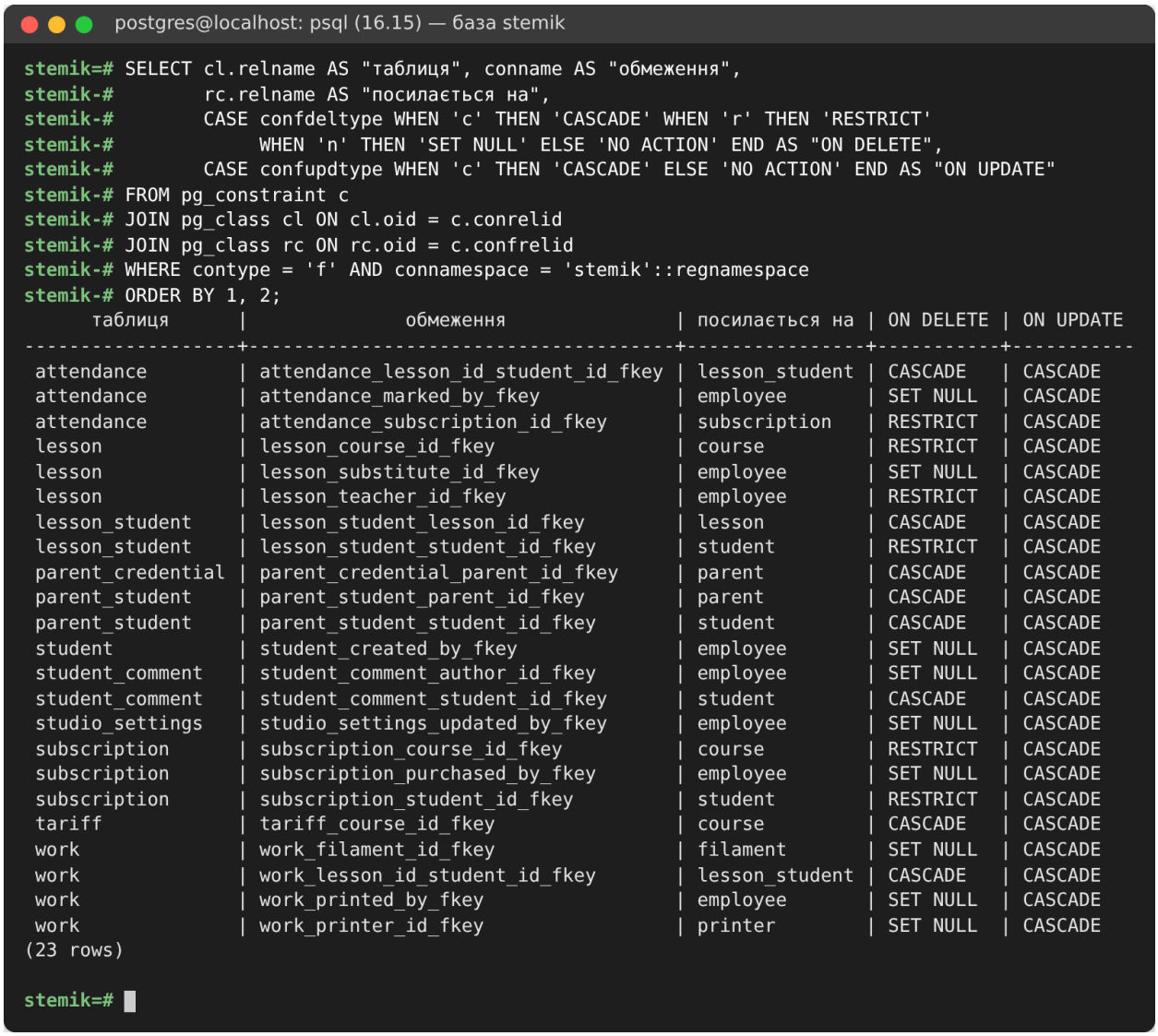


Рисунок 2.6 – Усі зовнішні ключі схеми з правилами ON DELETE / ON UPDATE (з системного каталогу pg_constraint)

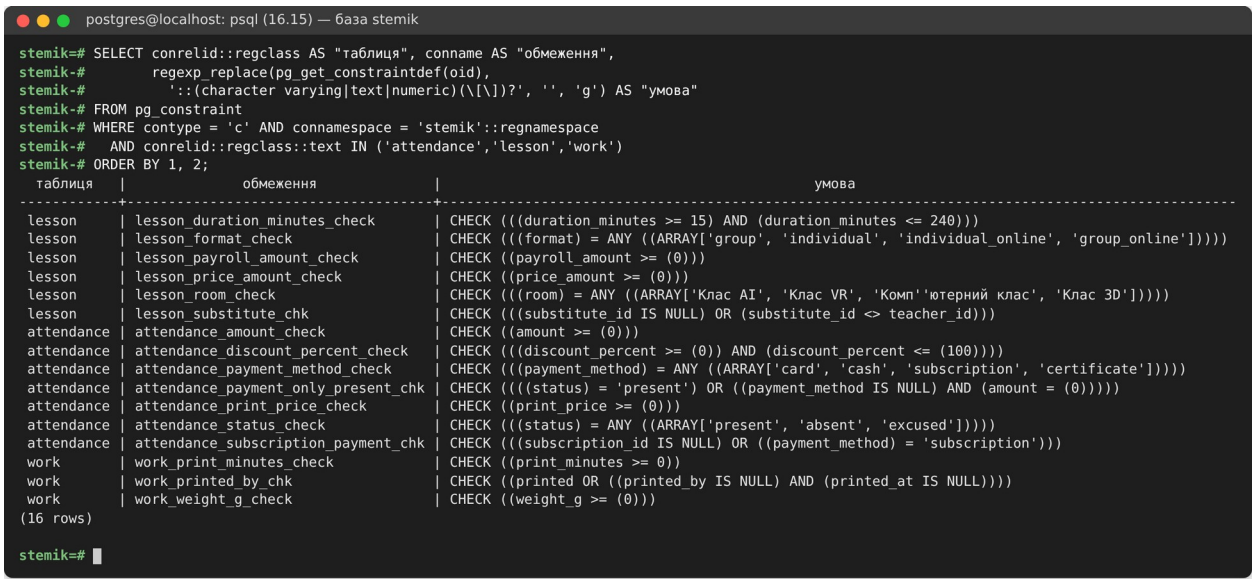


Рисунок 2.7 – CHECK-обмеження таблиць lesson, attendance, work

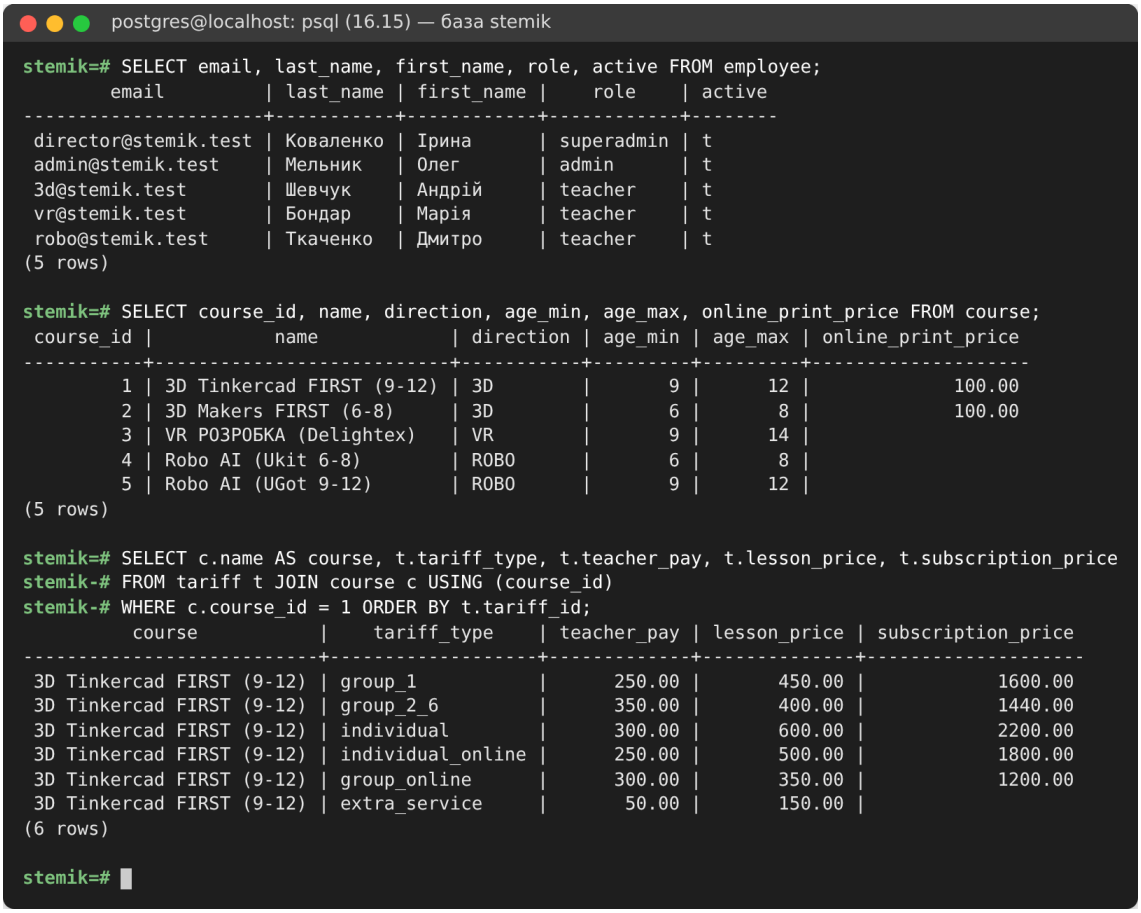


Рисунок 2.8 – Дані довідників: співробітники, курси, тарифи курсу «3D Tinkercad FIRST»

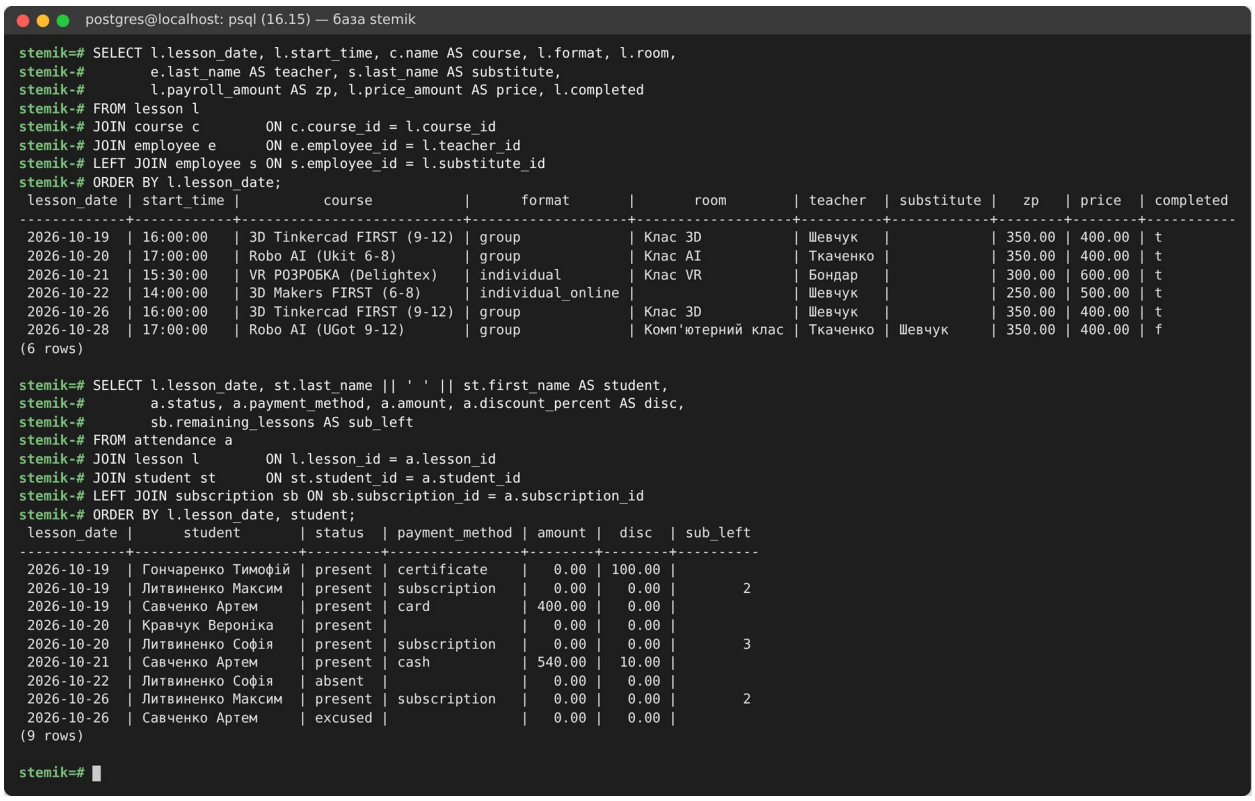


Рисунок 2.9 – Заняття з викладачами й заміною та журнал відвідувань з оплатами і залишком абонемента

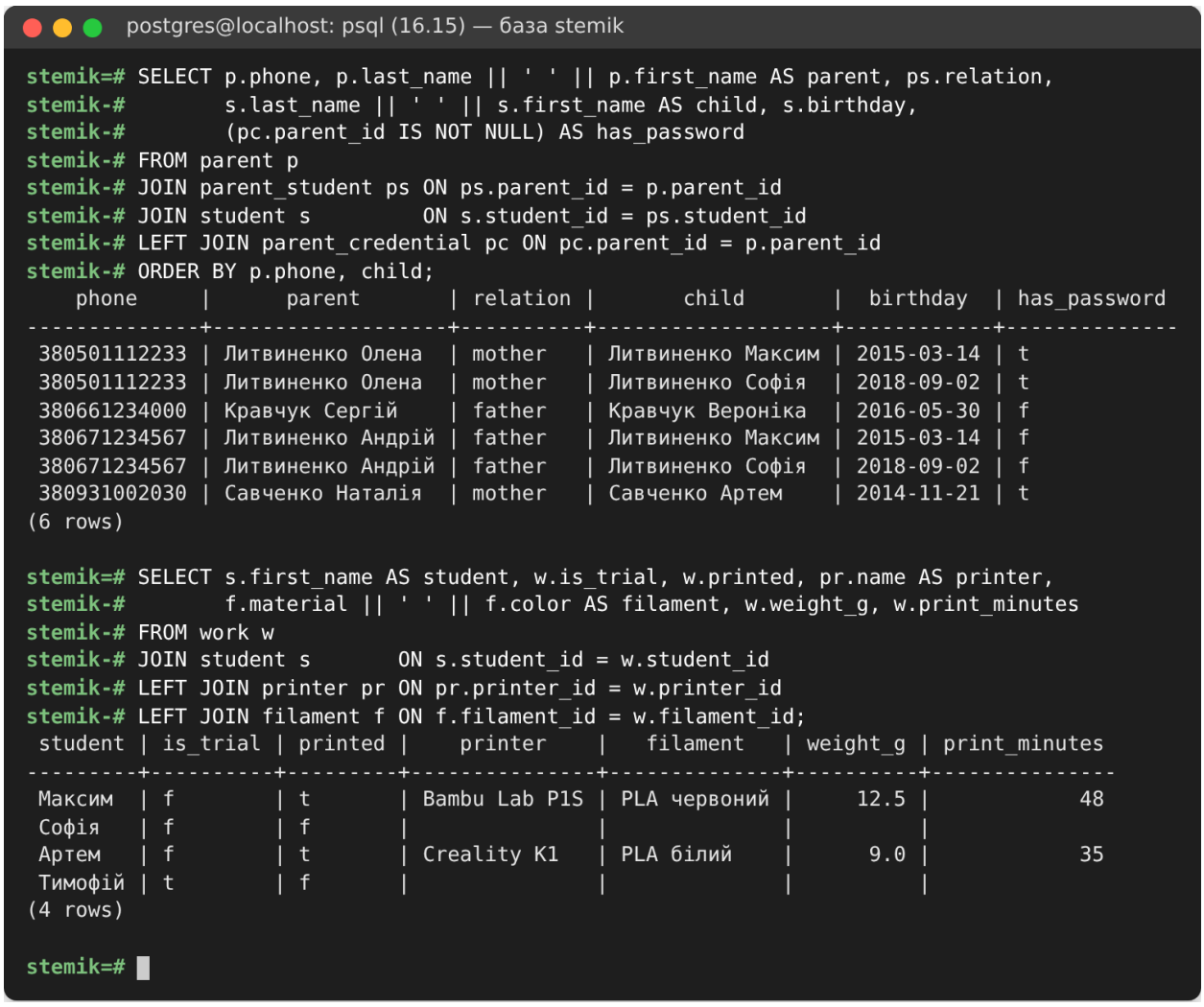


Рисунок 2.10 – Зв'язок M:N батьки–діти з ознакою наявності пароля (1:1) та роботи учнів з даними 3D-друку

Для перевірки обмежень цілісності виконано скрипт `03_constraint_tests.sql` (рис. 2.11): команди 1–7 навмисно порушують правила і відхиляються СУБД з назвою порушеного обмеження; тести 8–10 у транзакціях з `ROLLBACK` демонструють `ON DELETE CASCADE` (видалення заняття забрало 3 записи, 3 відмітки й 3 роботи), `ON DELETE SET NULL` (робота зберіглась без принтера) і значення `DEFAULT`.

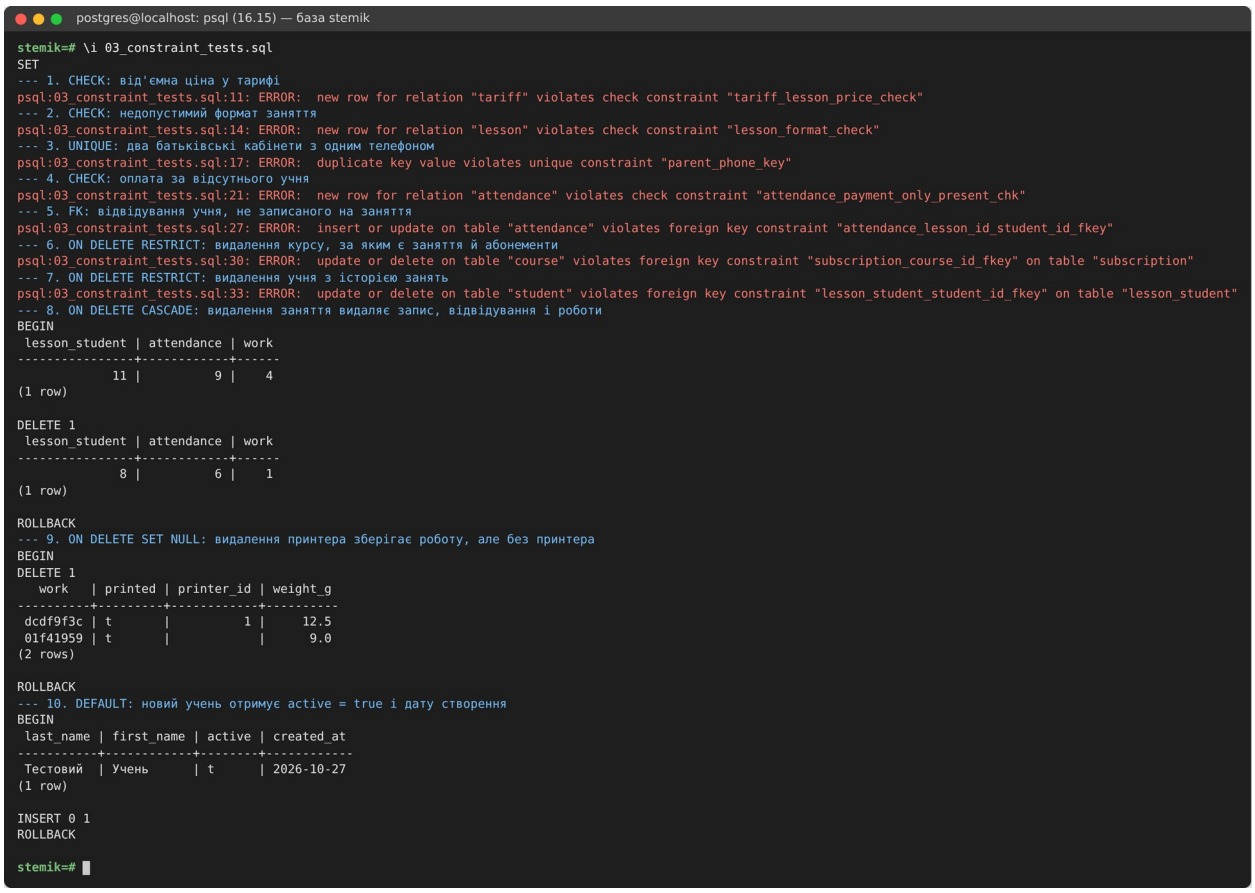


Рисунок 2.11 – Перевірка обмежень CHECK, UNIQUE, FOREIGN KEY та каскадних дій

Висновки до лабораторної роботи № 2

На основі ER-моделі з лабораторної роботи № 1 побудовано фізичну модель бази даних STEM-студії в СУБД PostgreSQL 16: 16 таблиць, 23 зовнішні ключі, 2 асоціативні таблиці для зв'язків M:N та 3 зв'язки 1:1. Для кожного атрибута обґрунтовано тип даних (точний `NUMERIC` для грошей, `UUID` для даних, що створюються в офлайн-режимі, компактні `SMALLINT` для довідників), визначено сурогатні первинні ключі та природні ключі як `UNIQUE`. Встановлено обмеження `NOT NULL`, `UNIQUE`, `CHECK` (у тому числі міжстовпцеві, що реалізують бізнес-правила оплати) та `DEFAULT`, а для кожного FK – обґрунтоване правило каскадної дії. DDL-скрипт виконано, базу заповнено тестовими даними, коректність обмежень і каскадних дій підтверджено тестовим скриптом. Порівняно з робочою схемою проєкту нова

модель усуває масив `student_ids`, широку таблицю тарифів і дубльовані атрибути, тому може слугувати планом рефакторингу продуктивної бази.

ДОДАТОК А. Повний DDL-скрипт фізичної схеми

Файл `docs/db-labs/sql/01_schema.sql`.

```
-- ========================================================================
-- Лабораторна робота №2. Фізична модель БД «STEM-студія "Стемік"»
-- СУБД: PostgreSQL 16 (у робочому проєкті — PostgreSQL у Supabase)
--
-- Порядок створення: спочатку довідники та «батьківські» таблиці,
-- потім таблиці з зовнішніми ключами, асоціативні таблиці — останніми.
-- Скрипт ідемпотентний: перед створенням видаляє схему stemik цілком.
-- ========================================================================

DROP SCHEMA IF EXISTS stemik CASCADE;
CREATE SCHEMA stemik;
SET search_path TO stemik;

-- ------------------------------------------------------------------------
-- 1. Співробітник (у проєкті — public.profiles)
-- ------------------------------------------------------------------------
CREATE TABLE employee (
    employee_id  UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
    email        VARCHAR(255) NOT NULL UNIQUE,
    last_name    VARCHAR(50)  NOT NULL,
    first_name   VARCHAR(50)  NOT NULL,
    role         VARCHAR(20)  NOT NULL DEFAULT 'teacher'
                 CHECK (role IN ('teacher', 'admin', 'superadmin')),
    active       BOOLEAN      NOT NULL DEFAULT TRUE,
    photo_url    TEXT,
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT now()
);

-- ------------------------------------------------------------------------
-- 2. Курс (напрямок навчання; у проєкті — текстове поле course)
-- ------------------------------------------------------------------------
CREATE TABLE course (
    course_id          SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name               VARCHAR(60)   NOT NULL UNIQUE,
    direction          VARCHAR(10)   NOT NULL
                       CHECK (direction IN ('3D', 'VR', 'ROBO')),
    age_min            SMALLINT      NOT NULL CHECK (age_min BETWEEN 4 AND 18),
    age_max            SMALLINT      NOT NULL CHECK (age_max BETWEEN 4 AND 18),
    online_print_price NUMERIC(10,2) CHECK (online_print_price >= 0),
    active             BOOLEAN       NOT NULL DEFAULT TRUE,
    CONSTRAINT course_age_range_chk CHECK (age_min <= age_max)
);

-- ------------------------------------------------------------------------
-- 3. Тариф (рядок «ЗП / ЦІНА» для пари курс × формат;
--    у проєкті — широка таблиця rate_cards із 30+ стовпцями)
-- ------------------------------------------------------------------------
CREATE TABLE tariff (
    tariff_id          INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    course_id          SMALLINT      NOT NULL
                       REFERENCES course(course_id)
                       ON DELETE CASCADE ON UPDATE CASCADE,
    tariff_type        VARCHAR(20)   NOT NULL
                       CHECK (tariff_type IN ('group_1', 'group_2_6', 'individual',
                                              'individual_online', 'group_online',
                                              'extra_service')),
    teacher_pay        NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (teacher_pay >= 0),
    lesson_price       NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (lesson_price >= 0),
    subscription_price NUMERIC(10,2) CHECK (subscription_price >= 0),
    UNIQUE (course_id, tariff_type)
);

-- ------------------------------------------------------------------------
-- 4. Батьки (обліковий запис батьківського кабінету; у проєкті —
--    parent_accounts)
-- ------------------------------------------------------------------------
CREATE TABLE parent (
    parent_id      UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    phone          VARCHAR(15) NOT NULL UNIQUE CHECK (phone ~ '^[0-9]{10,15}$'),
    last_name      VARCHAR(50),
    first_name     VARCHAR(50),
    active         BOOLEAN     NOT NULL DEFAULT TRUE,
    last_login_at  TIMESTAMPTZ,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
);

-- 4a. Облікові дані батьків — зв'язок 1:1 з parent (parent_credentials)
CREATE TABLE parent_credential (
    parent_id       UUID        PRIMARY KEY
                    REFERENCES parent(parent_id)
                    ON DELETE CASCADE ON UPDATE CASCADE,
    password_hash   TEXT        NOT NULL,
    password_set_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    failed_attempts SMALLINT    NOT NULL DEFAULT 0 CHECK (failed_attempts >= 0),
    locked_until    TIMESTAMPTZ
);

-- ---------------------------------------------------------------------
-- 5. Учень (students)
-- ---------------------------------------------------------------------
CREATE TABLE student (
    student_id  UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    last_name   VARCHAR(50) NOT NULL,
    first_name  VARCHAR(50) NOT NULL,
    birthday    DATE        CHECK (birthday BETWEEN DATE '2005-01-01' AND CURRENT_DATE),
    phone       VARCHAR(20),
    notes       TEXT,
    photo_url   TEXT,
    active      BOOLEAN     NOT NULL DEFAULT TRUE,
    created_by  UUID        REFERENCES employee(employee_id)
                ON DELETE SET NULL ON UPDATE CASCADE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 5a. Батьки ↔ Учень (M:N; parent_students)
CREATE TABLE parent_student (
    parent_id   UUID        NOT NULL REFERENCES parent(parent_id)
                ON DELETE CASCADE ON UPDATE CASCADE,
    student_id  UUID        NOT NULL REFERENCES student(student_id)
                ON DELETE CASCADE ON UPDATE CASCADE,
    relation    VARCHAR(20) NOT NULL DEFAULT 'parent'
                CHECK (relation IN ('mother', 'father', 'parent', 'guardian')),
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (parent_id, student_id)
);

-- ---------------------------------------------------------------------
-- 6. Абонемент (subscriptions) — пакет із N занять одного курсу
-- ---------------------------------------------------------------------
CREATE TABLE subscription (
    subscription_id   UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    student_id        UUID          NOT NULL REFERENCES student(student_id)
                      ON DELETE RESTRICT ON UPDATE CASCADE,
    course_id         SMALLINT      NOT NULL REFERENCES course(course_id)
                      ON DELETE RESTRICT ON UPDATE CASCADE,
    total_lessons     SMALLINT      NOT NULL DEFAULT 4 CHECK (total_lessons > 0),
    remaining_lessons SMALLINT      NOT NULL DEFAULT 4,
    total_price       NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (total_price >= 0),
    payment_method    VARCHAR(10)   CHECK (payment_method IN ('card', 'cash')),
    status            VARCHAR(10)   NOT NULL DEFAULT 'active'
                      CHECK (status IN ('active', 'completed', 'cancelled')),
    purchased_at      DATE          NOT NULL DEFAULT CURRENT_DATE,
    purchased_by      UUID          REFERENCES employee(employee_id)
                      ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT subscription_remaining_chk CHECK (remaining_lessons BETWEEN 0 AND total_lessons)
);

-- ---------------------------------------------------------------------
-- 7. Заняття (lessons)
-- ---------------------------------------------------------------------
CREATE TABLE lesson (
    lesson_id        UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    course_id        SMALLINT      NOT NULL REFERENCES course(course_id)
                     ON DELETE RESTRICT ON UPDATE CASCADE,
    teacher_id       UUID          NOT NULL REFERENCES employee(employee_id)
                     ON DELETE RESTRICT ON UPDATE CASCADE,
    substitute_id    UUID          REFERENCES employee(employee_id)
                     ON DELETE SET NULL ON UPDATE CASCADE,
    series_id        UUID,
    lesson_date      DATE          NOT NULL,
    start_time       TIME,
    duration_minutes SMALLINT      NOT NULL DEFAULT 60
                     CHECK (duration_minutes BETWEEN 15 AND 240),
    format           VARCHAR(20)   NOT NULL DEFAULT 'group'
```

```
                    CHECK (format IN ('group', 'individual',
                                      'individual_online', 'group_online')),
    room            VARCHAR(30)   CHECK (room IN ('Клас AI', 'Клас VR',
                                         'Комп''ютерний клас', 'Клас 3D')),
    theme           VARCHAR(200)  NOT NULL DEFAULT '',
    extra_service   BOOLEAN       NOT NULL DEFAULT FALSE,
    print_default   BOOLEAN       NOT NULL DEFAULT FALSE,
    completed       BOOLEAN       NOT NULL DEFAULT FALSE,
    payroll_amount  NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (payroll_amount >= 0),
    price_amount    NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (price_amount >= 0),
    CONSTRAINT lesson_substitute_chk CHECK (substitute_id IS NULL OR substitute_id <> teacher_id)
);

-- 7a. Заняття ↔ Учень — запис учня на заняття (M:N;
--     у проєкті — масив lessons.student_ids, що порушує 1НФ)
CREATE TABLE lesson_student (
    lesson_id   UUID NOT NULL REFERENCES lesson(lesson_id)
                ON DELETE CASCADE ON UPDATE CASCADE,
    student_id  UUID NOT NULL REFERENCES student(student_id)
                ON DELETE RESTRICT ON UPDATE CASCADE,
    PRIMARY KEY (lesson_id, student_id)
);

-- ---------------------------------------------------------------------
-- 8. Відвідування та оплата (lesson_attendance) — 1:1 до запису на заняття
-- ---------------------------------------------------------------------
CREATE TABLE attendance (
    attendance_id    UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    lesson_id        UUID          NOT NULL,
    student_id       UUID          NOT NULL,
    status           VARCHAR(10)   NOT NULL
                     CHECK (status IN ('present', 'absent', 'excused')),
    payment_method   VARCHAR(15)   CHECK (payment_method IN
                                   ('card', 'cash', 'subscription', 'certificate')),
    amount           NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (amount >= 0),
    discount_percent NUMERIC(5,2)  NOT NULL DEFAULT 0
                     CHECK (discount_percent BETWEEN 0 AND 100),
    subscription_id  UUID          REFERENCES subscription(subscription_id)
                     ON DELETE RESTRICT ON UPDATE CASCADE,
    print_enabled    BOOLEAN       NOT NULL DEFAULT FALSE,
    print_price      NUMERIC(10,2) CHECK (print_price >= 0),
    marked_by        UUID          REFERENCES employee(employee_id)
                     ON DELETE SET NULL ON UPDATE CASCADE,
    marked_at        TIMESTAMPTZ   NOT NULL DEFAULT now(),
    UNIQUE (lesson_id, student_id),
    FOREIGN KEY (lesson_id, student_id)
        REFERENCES lesson_student(lesson_id, student_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    -- оплата можлива лише для присутнього учня
    CONSTRAINT attendance_payment_only_present_chk CHECK (status = 'present' OR (payment_method
IS NULL AND amount = 0)),
    -- посилання на абонемент — лише при оплаті абонементом
    CONSTRAINT attendance_subscription_payment_chk CHECK (subscription_id IS NULL OR
payment_method = 'subscription')
);

-- ---------------------------------------------------------------------
-- 9. Обладнання й матеріали для 3D-друку (printers, filaments)
-- ---------------------------------------------------------------------
CREATE TABLE printer (
    printer_id            SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name                  VARCHAR(60)  NOT NULL UNIQUE,
    power_watts           NUMERIC(6,1) NOT NULL DEFAULT 0 CHECK (power_watts >= 0),
    depreciation_per_hour NUMERIC(8,2) NOT NULL DEFAULT 0
                          CHECK (depreciation_per_hour >= 0),
    active                BOOLEAN      NOT NULL DEFAULT TRUE
);

CREATE TABLE filament (
    filament_id  SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    material     VARCHAR(20)  NOT NULL,
    color        VARCHAR(30)  NOT NULL DEFAULT '',
    price_per_kg NUMERIC(8,2) NOT NULL DEFAULT 0 CHECK (price_per_kg >= 0),
    active       BOOLEAN      NOT NULL DEFAULT TRUE,
    UNIQUE (material, color)
);

-- ---------------------------------------------------------------------
-- 10. Робота учня (works) — фото моделі та дані про її 3D-друк
-- ---------------------------------------------------------------------
```

```
CREATE TABLE work (
    work_id       UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
    lesson_id     UUID         NOT NULL,
    student_id    UUID         NOT NULL,
    photo_url     TEXT,
    is_trial      BOOLEAN      NOT NULL DEFAULT FALSE,
    printed       BOOLEAN      NOT NULL DEFAULT FALSE,
    printed_by    UUID         REFERENCES employee(employee_id)
                  ON DELETE SET NULL ON UPDATE CASCADE,
    printed_at    TIMESTAMPTZ,
    printer_id    SMALLINT     REFERENCES printer(printer_id)
                  ON DELETE SET NULL ON UPDATE CASCADE,
    filament_id   SMALLINT     REFERENCES filament(filament_id)
                  ON DELETE SET NULL ON UPDATE CASCADE,
    weight_g      NUMERIC(7,1) CHECK (weight_g >= 0),
    print_minutes SMALLINT     CHECK (print_minutes >= 0),
    UNIQUE (lesson_id, student_id),
    FOREIGN KEY (lesson_id, student_id)
        REFERENCES lesson_student(lesson_id, student_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT work_printed_by_chk CHECK (printed OR (printed_by IS NULL AND printed_at IS NULL))
);

-- ---------------------------------------------------------------------
-- 11. Коментар до учня (student_comments)
-- ---------------------------------------------------------------------
CREATE TABLE student_comment (
    comment_id  UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    student_id  UUID        NOT NULL REFERENCES student(student_id)
                ON DELETE CASCADE ON UPDATE CASCADE,
    author_id   UUID        REFERENCES employee(employee_id)
                ON DELETE SET NULL ON UPDATE CASCADE,
    body        TEXT        NOT NULL CHECK (length(trim(body)) > 0),
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ---------------------------------------------------------------------
-- 12. Налаштування студії — таблиця з рівно одним рядком (studio_settings)
-- ---------------------------------------------------------------------
CREATE TABLE studio_settings (
    id                     BOOLEAN       PRIMARY KEY DEFAULT TRUE CHECK (id),
    calc_start_date        DATE,
    electricity_tariff     NUMERIC(6,2)  NOT NULL DEFAULT 0
                           CHECK (electricity_tariff >= 0),
    filament_waste_percent NUMERIC(5,2)  NOT NULL DEFAULT 0
                           CHECK (filament_waste_percent BETWEEN 0 AND 100),
    updated_by             UUID          REFERENCES employee(employee_id)
                           ON DELETE SET NULL ON UPDATE CASCADE,
    updated_at             TIMESTAMPTZ   NOT NULL DEFAULT now()
);

-- ---------------------------------------------------------------------
-- Індекси на зовнішні ключі, за якими найчастіше шукає застосунок
-- ---------------------------------------------------------------------
CREATE INDEX idx_lesson_date            ON lesson(lesson_date);
CREATE INDEX idx_lesson_teacher         ON lesson(teacher_id);
CREATE INDEX idx_lesson_student_student ON lesson_student(student_id);
CREATE INDEX idx_subscription_student   ON subscription(student_id, course_id);
CREATE INDEX idx_parent_student_student ON parent_student(student_id);
CREATE INDEX idx_comment_student        ON student_comment(student_id);

-- ---------------------------------------------------------------------
-- Коментарі до таблиць (відображаються у psql \dt+ та в pgAdmin)
-- ---------------------------------------------------------------------
COMMENT ON TABLE employee          IS 'Співробітники студії: викладачі та адміністратори';
COMMENT ON TABLE course            IS 'Курси (напрямки навчання)';
COMMENT ON TABLE tariff            IS 'Тарифи: ЗП викладача та ціна для клієнта за курсом і
форматом';
COMMENT ON TABLE parent            IS 'Батьки — власники батьківського кабінету';
COMMENT ON TABLE parent_credential IS 'Пароль і блокування входу до кабінету батьків (1:1)';
COMMENT ON TABLE student           IS 'Учні студії';
COMMENT ON TABLE parent_student    IS 'Зв''язок M:N батьки — діти';
COMMENT ON TABLE subscription      IS 'Абонементи на пакет занять';
COMMENT ON TABLE lesson            IS 'Заняття за розкладом';
COMMENT ON TABLE lesson_student    IS 'Запис учня на заняття (M:N)';
COMMENT ON TABLE attendance        IS 'Відвідування (П/Н/ВПП) та оплата заняття';
COMMENT ON TABLE printer           IS '3D-принтери';
COMMENT ON TABLE filament          IS 'Пластик (філамент) для 3D-друку';
COMMENT ON TABLE work              IS 'Роботи (моделі) учнів та їх друк';
COMMENT ON TABLE student_comment   IS 'Коментарі викладачів про учня';
```

```
COMMENT ON TABLE studio_settings   IS 'Глобальні налаштування студії (один рядок)';
```